

Reviewer Pack — Adjacency Spectrum Log-Discriminant Tracks Max-Cut SDP Slack

Entry 2deb9c6539ac · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 19:02:21 UTC

Adjacency Spectrum Log-Discriminant Tracks Max-Cut SDP Slack

Entry ID: 2deb9c6539ac **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 19:02:21 UTC

1. Conjecture

Field A (mathematical branch): Real algebraic geometry: Hermite-Vandermonde discriminant of a real-rooted characteristic polynomial and Fekete-Szegő logarithmic capacity on the real line **Field B** (complexity object): Degree-2 SOS (Goemans-Williamson / Delorme-Poljak) spectral upper bound for max-CUT, viewed as a moment-matrix invariant relevant to algebraic proof systems related to Extended Frege

Statement:

For a graph G on n vertices with adjacency eigenvalues $\lambda_1 \leq \dots \leq \lambda_n$, define $\text{LogDisc}(G) := (1/n^2) * \sum_{i < j} \log|\lambda_i - \lambda_j|$ (the normalized Hermite-Vandermonde log-discriminant of the real-rooted characteristic polynomial $\det(tI - A)$) and define the degree-2 SOS spectral slack $S(G) := (|E|/2 - (n/4)\lambda_1 - \text{MaxCut}(G))/n$. *Conjecture: over independent uniformly random 3-regular graphs at each fixed n in $\{12, 16, 20\}$, the Pearson correlation $\rho(S, \text{LogDisc})$ satisfies $\rho \geq 0.4$, and there exist universal constants a in $(0, 5)$, b in $\mathbb{R}$ such that $|E[S] - aE[\text{LogDisc}] - b| \leq 0.05$ for all such n . A single random instance with $S(G) > 0.2$ while $\text{LogDisc}(G) \leq -1$ falsifies the conjecture.*

Rationale (proposer's reasoning):

The Delorme-Poljak spectral bound for max-CUT is tight precisely when the adjacency spectrum is concentrated near its extreme eigenvalue, whereas the Hermite-Vandermonde log-discriminant is the real-algebraic obstruction to root coalescence (Fekete-Szegő logarithmic energy on the real line). Linking the SOS spectral slack to a classical RAG invariant of the characteristic polynomial sidesteps ring-extension arithmetization (so it should evade algebrization) and avoids any single-truth-table predicate (so it should evade natural proofs). The invariant is relational (compares pairs of eigenvalues), not a single-number truth-table statistic.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f17ba945e2ed47f8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 i.i.d. random 3-regular graphs per n in $\{12, 16, 20\}$, Pearson $\rho(S, \text{LogDisc}) \geq 0.4$ at every n , and a single OLS fit $S = a \cdot \text{LogDisc} + b$ (a in $(0, 5)$, b real) yields $|E[S]_n - a \cdot E[\text{LogDisc}]_n - b| \leq 0.05$ at all three n . Any instance with $S > 0.2$ and $\text{LogDisc} \leq -1$ falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 10 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -max-cut SDP spectral bound Delorme Poljak eigenvalue slack random regular graph - Hermite discriminant characteristic polynomial graph eigenvalues logarithmic capacity Fekete-Szego - degree-2 sum-of-squares Goemans-Williamson moment matrix adjacency spectrum random 3-regular

Top relevant hits considered: - [http://arxiv.org/abs/1411.1698v2] On the Max-Cut of Sparse Random Graphs - [http://arxiv.org/abs/2109.02238v1] Optimal solutions and ranks in the max-cut SDP - [http://arxiv.org/abs/1207.5696v1] Beyond Max-Cut: λ -Extendible Properties Parameterized Above the Poljak-Turzík Bound - [http://arxiv.org/abs/1801.09531v2] Fekete-Szego Inequality for Analytic and Bi-univalent Functions Subordinate to Chebyshev Polynomials - [http://arxiv.org/abs/1806.07358v1] No Threshold graphs are cospectral - [http://arxiv.org/abs/2207.13180v3] Hermite trace polynomials and chaos decompositions for the Hermitian Brownian motion - [http://arxiv.org/abs/1005.0979v1] Supersymmetry in Random Matrix Theory - [http://arxiv.org/abs/2007.12819] Support of Closed Walks and Second Eigenvalue Multiplicity of the Normalized Adjacency Matrix

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3regular_graph(n):
        while True:
            edges = set()
```

```

    for i in range(n):
        neighbors = random.sample(range(n), 2)
        if (i, neighbors[0]) not in edges and (neighbors[0], i) not in edges:
            edges.add((i, neighbors[0]))
            edges.add((i, neighbors[1]))
    if len(edges) == n * 3 // 2:
        return [set() for _ in range(n)], list(edges)

def adjacency_matrix_to_eigenvalues(A):
    n = len(A)
    eigenvalues = []
    for i in range(100): # Max 100 iterations to avoid infinite loop
        v = [random.gauss(0, 1) for _ in range(n)]
        v /= math.sqrt(sum(x * x for x in v))
        Av = [sum(A[i][j] * v[j] for j in range(n)) for i in range(n)]
        lambda_i = sum(v[i] * Av[i] for i in range(n)) / sum(v[i] * v[i] for i in range(n))
        eigenvalues.append(lambda_i)
    return sorted(eigenvalues)

def max_cut(G):
    n = len(G)
    best_cut_size = 0
    for mask in range(1 << n):
        cut_size = sum((mask >> i) & 1 for i in range(n))
        if cut_size > best_cut_size:
            left = [i for i in range(n) if (mask >> i) & 1]
            right = [i for i in range(n) if not (mask >> i) & 1]
            cut_size = sum(1 for u, v in G if ((u in left and v in right) or (v in left and u
            best_cut_size = cut_size
    return best_cut_size

def log_disc(eigenvalues):
    n = len(eigenvalues)
    total = 0
    for i in range(n):
        for j in range(i + 1, n):
            gap = abs(eigenvalues[i] - eigenvalues[j])
            if gap > 1e-12:
                total += math.log(gap)
    return total / (n * n)

def ols_regression(x, y):
    n = len(x)
    sum_x = sum(x)
    sum_y = sum(y)
    sum_xy = sum(xi * yi for xi, yi in zip(x, y))
    sum_xx = sum(xi * xi for xi in x)
    a = (n * sum_xy - sum_x * sum_y) / (n * sum_xx - sum_x ** 2)
    b = (sum_y - a * sum_x) / n
    return a, b

def pearson_correlation(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov_xy = sum((xi - mean_x) * (yi - mean_y) for xi, yi in zip(x, y)) / n

```

```

std_x = math.sqrt(sum((xi - mean_x) ** 2 for xi in x) / n)
std_y = math.sqrt(sum((yi - mean_y) ** 2 for yi in y) / n)
return cov_xy / (std_x * std_y)

n_values = [12, 16, 20]
results = []

for n in n_values:
    G, edges = generate_3regular_graph(n)
    A = [[0] * n for _ in range(n)]
    for u, v in edges:
        A[u][v] = A[v][u] = 1

    eigenvalues = adjacency_matrix_to_eigenvalues(A)
    max_cut_value = max_cut(G)
    log_disc_value = log_disc(eigenvalues)

    results.append({
        "n": n,
        "log_disc": log_disc_value,
        "max_cut": max_cut_value,
        "sos_slack": (len(edges) / 2 - (n / 4) * eigenvalues[0] - max_cut_value) / n
    })

if any(result["sos_slack"] > 0.2 and result["log_disc"] <= -1 for result in results):
    return {
        "metric_name": "SOS Slack vs Log-Discriminant",
        "metric_value": None,
        "instances_tested": len(results),
        "conjecture_holds": False,
        "counterexample": "Found instance with SOS Slack > 0.2 and Log-Discriminant <= -1"
    }

log_disc_values = [result["log_disc"] for result in results]
sos_slack_values = [result["sos_slack"] for result in results]

rho = pearson_correlation(log_disc_values, sos_slack_values)
a, b = ols_regression(log_disc_values, sos_slack_values)

return {
    "metric_name": "SOS Slack vs Log-Discriminant",
    "metric_value": rho,
    "instances_tested": len(results),
    "conjecture_holds": rho >= 0.4 and abs(a * Fraction(12) + b - Fraction(12, 1)) <= 0.05 and
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(2, 33)]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

log_disc_values = [run_trial(seed)["log_disc"] for seed in seeds]

```

```

sos_slack_values = [run_trial(seed)["sos_slack"] for seed in seeds]

rho = pearson_correlation(log_disc_values, sos_slack_values)
a, b = ols_regression(log_disc_values, sos_slack_values)

support_fraction = sum(1 for seed in seeds if run_trial(seed)["conjecture_holds"]) / len(seeds)

if all(run_trial(seed)["conjecture_holds"] for seed in seeds):
    print(f"RESULT: SUPPORTED mean={rho} std={math.sqrt(sum((x - rho) ** 2 for x in log_disc_v
elif any(result["sos_slack"] > 0.2 and result["log_disc"] <= -1 for result in (run_trial(seed)
    print(f"RESULT: FALSIFIED counterexample='Found instance with SOS Slack > 0.2 and Log-Disc
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence support_fraction={support_fractions}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ed02682.py", line 132, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ed02682.py", line 93, in run_trial
    eigenvalues = adjacency_matrix_to_eigenvalues(A)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ed02682.py", line 37, in adjacency_matrix_to_eigenvalues
    v /= math.sqrt(sum(x * x for x in v))
TypeError: unsupported operand type(s) for /=: 'list' and 'float'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `adjacency_matrix_to_eigenvalues` (`list /= float`) before producing any data, so neither the support nor the falsification condition can be evaluated. | next: Fix the eigenvector normalization bug (convert `v` to a numpy array or use list comprehension division) and rerun the 90-graph trial to obtain valid Pearson correlations and OLS residuals.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	175666
2	preregistration	claude_max	opus	0	0	6741
3	novelty	claude_max	opus	0	0	4242
4	novelty	claude_max	opus	0	0	8634
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19864
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17064
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16512
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19497
9	judge	claude_max	opus	0	0	8095

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 276314 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/2deb9c6539ac.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2deb9c6539ac.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2deb9c6539ac.tar.gz` (if generated)